

Direct Memory Access

1 Basic Idea

Definition

Direct Memory Access (DMA) is an I/O technique where a DMA controller transfers data between an I/O device and main memory without the CPU copying every byte or word.

Without DMA, the CPU must repeatedly read data from a device controller register and store it in memory. This wastes CPU time, especially for large transfers such as disk blocks.

Main Benefit

DMA lets the CPU start an I/O transfer and then do other work while the DMA controller moves the data.

2 DMA Controller

Role

The **DMA controller** has independent access to the system bus and controls the data transfer after the CPU programs it.

A DMA controller may be built into a device controller, or one shared DMA controller may handle several devices.

DMA register	Purpose
Memory address register	Stores the main-memory address where data should be read or written.
Byte count register	Stores how many bytes remain to transfer.
Control register	Stores direction, device or port, transfer unit, and burst size.

3 Disk Read Without DMA

Step	Action
1	Disk controller reads the disk block into its internal buffer.
2	Disk controller verifies the checksum.
3	Disk controller interrupts the CPU.
4	Operating system copies the block from controller buffer to main memory, one byte or word at a time.

Cost

The CPU is directly involved in the copy loop, so CPU time is spent moving data instead of running useful work.

4 Disk Read With DMA

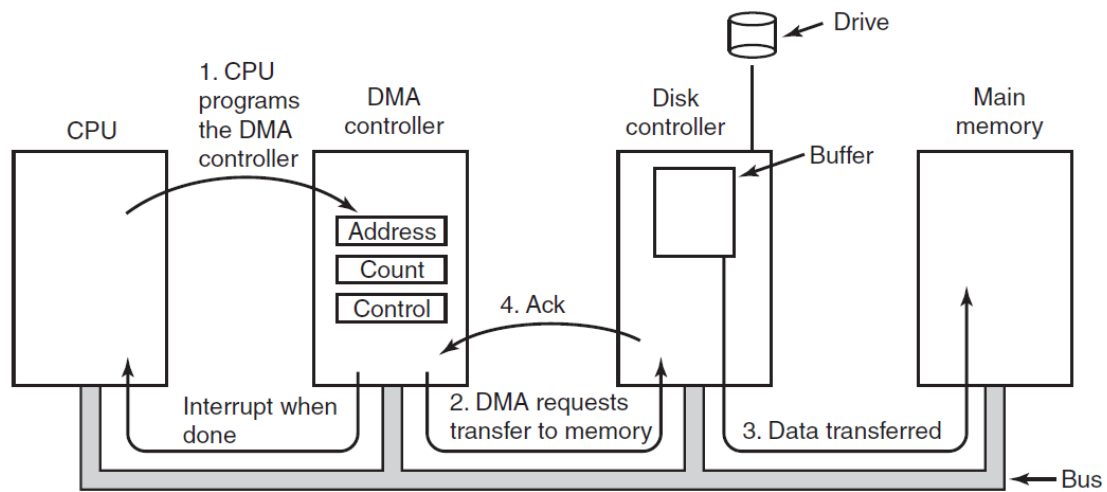


Figure 5-4. Operation of a DMA transfer.

Step	Action
1	CPU loads the DMA registers with address, count, direction, and device information.
2	Disk controller reads the disk block into its internal buffer and checks it.
3	DMA controller requests data from the disk controller.
4	Disk controller writes the data to main memory.
5	Disk controller acknowledges; DMA updates the address and byte count.
6	When the count reaches 0, DMA interrupts the CPU.

Result

When the operating system handles the final interrupt, the block is already in memory.

5 DMA Channels

Multiple Transfers

More advanced DMA controllers can handle multiple transfers using multiple internal register sets called channels.

Each channel stores the parameters for one transfer. Different channels must refer to different device controllers. The DMA controller chooses which pending device to serve next.

Scheduling method	Meaning
Round robin	Serve channels in cyclic order.
Priority based	Serve more important devices first.
Ack lines	Help distinguish acknowledgements from different DMA channels.

6 Transfer Modes

Mode	Description
Cycle stealing	DMA transfers one word at a time and uses occasional bus cycles. The CPU may be delayed slightly.
Burst mode	DMA acquires the bus, transfers many words, and then releases the bus. It is efficient but can block the CPU longer.

Trade-off

Cycle stealing gives better CPU responsiveness. Burst mode gives better transfer efficiency.

7 Fly-By Mode and Alternative Mode

Fly-By Mode

In **fly-by mode**, the DMA controller tells the device controller to transfer data directly to main memory.

Some DMA controllers use a two-step method instead:

1. Device controller sends a word to the DMA controller.
2. DMA controller writes that word to memory.

This needs an extra bus cycle per word, but it is more flexible. It can support device-to-device copies and memory-to-memory copies.

8 Physical and Virtual Addresses

Most DMA controllers use **physical memory addresses**. Therefore, the operating system must translate the target buffer's virtual address into a physical address before programming the DMA controller.

Address Translation

If a DMA controller accepts virtual addresses, it must use the MMU for translation. This is uncommon because the MMU is usually part of the CPU, not the memory system.

9 Why Devices Use Internal Buffers

Disk controllers usually read data into an internal buffer before DMA begins.

Reason	Explanation
Checksum first	The controller can verify the block before transferring it to memory.
Timing control	Disk bits arrive at a fixed rate, even if the system bus is busy.
Simpler design	DMA to memory is not time-critical once the full block is buffered.

Overflow Risk

Without enough buffering, a busy bus can prevent the controller from storing incoming disk data fast enough, causing an overflow error.

10 When DMA May Not Be Used

DMA is not always worth using. On some low-end or embedded systems, removing the DMA controller reduces cost. Also, if the CPU has no other work and is much faster than the DMA controller, CPU-driven I/O may be acceptable.

Final Point

DMA reduces CPU copying overhead, but it adds controller hardware, bus arbitration, address-translation, and buffering concerns.